

## MA3632 Lecture 5 – Supervised Learning: Framework and $k$ -Nearest Neighbours

The first four weeks of this module were concerned with data in its raw form: how it is acquired, cleaned, transformed, and compressed into a representation suitable for analysis. From this week we turn to the central problem of the module – learning from data. The goal is not merely to describe what a dataset contains, but to extract a function that generalises beyond the observed examples.

This lecture establishes the formal framework that underlies every supervised learning method we will study, then examines the  $k$ -nearest neighbour classifier in detail. It is a deliberately simple algorithm, but it makes the essential concepts – prediction, error, and the tension between fitting the training data and generalising to new data – concrete and geometric. The parametric models of Weeks 6 and 7 will be easier to understand once these ideas are in place.

### 1 The Supervised Learning Problem

**Definition 1.1.** A **supervised learning problem** consists of:

- (i) an **input space**  $\mathcal{X} \subseteq \mathbb{R}^p$ , whose elements are called *feature vectors*;
- (ii) an **output space**  $\mathcal{Y}$ ;
- (iii) an unknown probability distribution  $P$  on  $\mathcal{X} \times \mathcal{Y}$ ;
- (iv) a **training set**  $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ , where each  $(\mathbf{x}_i, y_i)$  is drawn independently from  $P$ .

The task is to use  $\mathcal{D}$  to construct a function  $\hat{f}: \mathcal{X} \rightarrow \mathcal{Y}$  that predicts  $y$  accurately for inputs  $\mathbf{x}$  not seen during training.

When  $\mathcal{Y} = \{1, \dots, C\}$  for some finite  $C$ , the problem is called *classification*; when  $\mathcal{Y} \subseteq \mathbb{R}$ , it is called *regression*. The distinction matters because it governs both which loss functions are natural and what it means for a prediction to be good. In classification, a prediction is either right or wrong (or costs different amounts depending on which error is made); in regression, predictions lie on a continuum and small errors are qualitatively different from large ones.

It is worth pausing on the assumption that each  $(\mathbf{x}_i, y_i)$  is drawn *independently* from the same distribution  $P$ . This is the i.i.d. assumption (*independent and identically distributed*), and it underlies most of the theory in this module. It fails in time series data, where observations are correlated across time, and in settings where the distribution of inputs changes between training and deployment – a phenomenon known as *dataset shift*. We will note where these complications arise, but for now we work within the i.i.d. framework.

#### 1.1 Loss functions and the Bayes predictor

**Definition 1.2.** A **loss function** is a map  $\ell: \mathcal{Y} \times \mathcal{Y} \rightarrow [0, \infty)$  that quantifies the cost of predicting  $\hat{y}$  when the true output is  $y$ . Common choices include:

- (i) **0–1 loss** for classification:  $\ell(y, \hat{y}) = \mathbf{1}[y \neq \hat{y}]$ ;
- (ii) **squared loss** for regression:  $\ell(y, \hat{y}) = (y - \hat{y})^2$ ;
- (iii) **absolute loss** for regression:  $\ell(y, \hat{y}) = |y - \hat{y}|$ .

The choice of loss function is not merely a convention; it determines which prediction is optimal. Under squared loss, the prediction that minimises expected loss at a given  $\mathbf{x}$  is the conditional mean  $\mathbb{E}[y \mid \mathbf{x}]$ . Under absolute loss, the optimal prediction is the conditional median. Under 0–1 loss, it is the conditional mode – the most probable class. A function that achieves the minimum possible expected loss is called a **Bayes predictor**, and the corresponding minimum risk is the **Bayes risk**  $R^*$ . No learning algorithm can do better than  $R^*$  in expectation; it represents the irreducible uncertainty in the problem due to the noise in  $P$ .

**Example 1.3.** Suppose  $\mathcal{Y} = \{0, 1\}$  and  $P(y = 1 \mid \mathbf{x}) = \eta(\mathbf{x})$ . Under 0–1 loss, the Bayes predictor assigns class 1 if  $\eta(\mathbf{x}) > 1/2$  and class 0 otherwise. The Bayes risk is  $R^* = \mathbb{E}_{\mathbf{x}}[\min(\eta(\mathbf{x}), 1 - \eta(\mathbf{x}))]$ , which is zero only when every point is deterministically one class or the other. When labels are genuinely noisy – when the same feature vector  $\mathbf{x}$  can map to either class –  $R^* > 0$  and no classifier can achieve zero generalisation error.

## 1.2 Empirical risk minimisation

The ideal criterion for evaluating  $\hat{f}$  is the **risk**:

$$R(\hat{f}) = \mathbb{E}_{(\mathbf{x}, y) \sim P}[\ell(y, \hat{f}(\mathbf{x}))].$$

Since  $P$  is unknown,  $R(\hat{f})$  cannot be computed directly. What we can compute is the **empirical risk**:

$$\hat{R}(\hat{f}) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{f}(\mathbf{x}_i)).$$

By the law of large numbers,  $\hat{R}(\hat{f}) \rightarrow R(\hat{f})$  as  $n \rightarrow \infty$  for any fixed  $\hat{f}$ . This motivates the following strategy.

**Definition 1.4. Empirical risk minimisation (ERM)** is the strategy of choosing  $\hat{f}$  from a specified **hypothesis class**  $\mathcal{H}$  to minimise the empirical risk:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \hat{R}(f).$$

This pointwise convergence of  $\hat{R}(f)$  to  $R(f)$  holds for any single, fixed  $f$ ; it does not by itself guarantee that the minimiser  $\hat{f}$  found by searching over  $\mathcal{H}$  also has small generalisation error, since  $\hat{f}$  is chosen using  $\mathcal{D}$  and is therefore not fixed independently of the data. When  $\mathcal{H}$  is large or flexible enough, controlling the complexity of  $\mathcal{H}$  – via uniform convergence bounds or other means – is essential for ERM to generalise; we return to this below.

The hypothesis class  $\mathcal{H}$  is the set of functions the learning algorithm is permitted to consider. Its choice encodes our assumptions about the problem. Some representative examples:

$\mathcal{H}_{\text{lin}} = \{f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b : \mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}\}$ , the class of affine functions. This is the hypothesis class underlying linear regression; logistic regression uses the same affine linear predictor but composes it with a logistic link function to produce class probabilities, so its hypothesis class is best understood as  $\mathcal{H}_{\text{lin}}$  passed through a fixed nonlinearity rather than  $\mathcal{H}_{\text{lin}}$  itself.

$\mathcal{H}_{\text{poly}, d}$ , the class of polynomial functions of degree at most  $d$ . For  $d = 1$  this reduces to  $\mathcal{H}_{\text{lin}}$ ; for  $d > 1$  the class is strictly larger and can represent curved decision boundaries.

$\mathcal{H}_{\text{tree}, L}$ , the class of decision trees with at most  $L$  leaves. This class partitions  $\mathcal{X}$  into at most  $L$  rectangular regions and assigns a constant prediction in each.

$\mathcal{H}_{k\text{-NN}}$ , implicitly defined by the  $k$ -NN algorithm: predictions are determined by local majority votes, and no explicit parametric form is assumed.

Many supervised learning algorithms can be understood as a particular choice of  $\mathcal{H}$  together with a method for minimising (or approximately minimising)  $\hat{R}$  over it; this is a useful unifying lens for the parametric models introduced from Week 6 onwards. Others, such as  $k$ -NN, are more naturally described algorithmically than as the solution to an explicit optimisation problem, even though  $\mathcal{H}_{k\text{-NN}}$  can formally be written down.

*Remark 1.5.* Minimising  $\hat{R}$  over a very large  $\mathcal{H}$  can drive the training error to zero while leaving the generalisation error high. Consider a hypothesis class rich enough to interpolate every training label exactly: such a class always admits a solution with  $\hat{R} = 0$ , but such a solution may have fitted noise in  $\mathcal{D}$  as well as signal, leading to poor generalisation. This is the phenomenon of *overfitting*. The gap  $R(\hat{f}) - \hat{R}(\hat{f})$  is called the *generalisation gap*, and controlling it – by restricting  $\mathcal{H}$ , by regularisation, or by early stopping – is one of the central concerns of learning theory.

## 2 Bias, Variance, and the Decomposition of Generalisation Error

The generalisation error of a learning algorithm depends on two things: how well the best function in  $\mathcal{H}$  approximates the true relationship, and how much the learned function fluctuates when  $\mathcal{D}$  changes. These two contributions are in tension, and understanding them separately is essential for diagnosing model behaviour and choosing between algorithms.

Fix a test point  $\mathbf{x}$  and suppose the true relationship is  $y = f^*(\mathbf{x}) + \varepsilon$ , where  $\varepsilon$  is zero-mean noise with variance  $\sigma^2$ , independent of  $\mathbf{x}$  and of the training set  $\mathcal{D}$ . Let  $\hat{f}_{\mathcal{D}}$  denote the function learned from a training set  $\mathcal{D}$  drawn from  $P$ , and write  $\bar{f}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})]$  for the average prediction across all possible training sets of size  $n$ .

**Theorem 2.1** (Bias–Variance Decomposition). *Under squared loss, the expected generalisation error at  $\mathbf{x}$ , taken jointly over the training set  $\mathcal{D}$  and the test-point noise  $\varepsilon$ , decomposes as*

$$\mathbb{E}_{\mathcal{D}, \varepsilon}[(y - \hat{f}_{\mathcal{D}}(\mathbf{x}))^2] = \underbrace{(f^*(\mathbf{x}) - \bar{f}(\mathbf{x}))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \bar{f}(\mathbf{x}))^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}.$$

*Proof.* Add and subtract  $f^*(\mathbf{x})$  and  $\bar{f}(\mathbf{x})$  to write

$$y - \hat{f}_{\mathcal{D}} = \underbrace{(y - f^*)}_{\varepsilon} + \underbrace{(f^* - \bar{f})}_{\text{const. w.r.t. } \mathcal{D} \text{ and } \varepsilon} + \underbrace{(\bar{f} - \hat{f}_{\mathcal{D}})}_{\text{mean zero over } \mathcal{D}}.$$

Square and take  $\mathbb{E}_{\mathcal{D}, \varepsilon}[\cdot]$ . Since  $\varepsilon$  is independent of  $\mathcal{D}$  with  $\mathbb{E}_{\varepsilon}[\varepsilon] = 0$ , and  $\mathbb{E}_{\mathcal{D}}[\bar{f} - \hat{f}_{\mathcal{D}}] = 0$  by definition of  $\bar{f}$ , each of the three cross-product terms involves a factor with zero expectation taken over the variable it is independent of, so all three vanish under the joint expectation. What remains is  $\mathbb{E}_{\varepsilon}[\varepsilon^2] = \sigma^2$ , the constant squared term  $(f^* - \bar{f})^2$ , and  $\mathbb{E}_{\mathcal{D}}[(\bar{f} - \hat{f}_{\mathcal{D}})^2]$ , giving the three terms stated.  $\square$

Each term has a clear interpretation. The **bias** measures the systematic error of the algorithm: how far, on average, the learned function is from the truth. A model that is too simple – say, a linear function fitted to data generated by a quadratic – will have high bias because no element of  $\mathcal{H}$  can approximate  $f^*$  well, regardless of how much data is available. The **variance** measures the sensitivity of the learned function to the particular training sample: a model that changes dramatically when a few points are replaced has high variance, even if its average prediction is

accurate. The **irreducible noise**  $\sigma^2$  is a property of  $P$  alone and cannot be reduced by any choice of model or algorithm.

The practical implication is that bias and variance pull in opposite directions as model complexity changes. Increasing complexity (larger  $\mathcal{H}$ , higher-degree polynomial, smaller  $k$  in  $k$ -NN) tends to reduce bias and increase variance. Decreasing complexity has the opposite effect. This often leads to the familiar U-shaped test-error curve as model complexity increases – high at both extremes and minimised at some intermediate value – although the pattern need not be exactly U-shaped in every problem. Identifying that intermediate value – whether by cross-validation, regularisation, or theoretical analysis – is a central task of model selection.

**Example 2.2.** Consider estimating a constant function  $f^*(\mathbf{x}) = \mu$  from  $n$  noisy observations  $y_i = \mu + \varepsilon_i$  with  $\varepsilon_i \sim N(0, \sigma^2)$ . The sample mean  $\bar{y}$  has bias zero and variance  $\sigma^2/n$ . The estimator that always predicts a fixed constant  $c$  has squared bias  $(\mu - c)^2$  and zero variance. The bias–variance decomposition shows that the sample mean has lower total expected squared error whenever  $\sigma^2/n < (\mu - c)^2$ , i.e. whenever the squared bias of the constant estimator exceeds its variance advantage. This simple example illustrates why neither zero bias nor zero variance is the right objective in isolation.

### 3 $k$ -Nearest Neighbours

The  $k$ -nearest neighbour algorithm ( $k$ -NN) is among the oldest and most transparent prediction methods. It makes no parametric assumptions about the form of  $f^*$  and requires no training phase in the conventional sense: the entire training set is retained, and predictions are made at query time by examining local neighbourhoods. For this reason  $k$ -NN is called a *lazy* or *instance-based* learner, in contrast to the *eager* learners – linear models, decision trees – that fit a compact set of parameters during training and discard the data afterwards.

#### 3.1 The algorithm

**Definition 3.1.** Let  $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$  with  $\mathcal{Y} = \{1, \dots, C\}$ . Fix a positive integer  $k \leq n$  and a distance function  $d: \mathcal{X} \times \mathcal{X} \rightarrow [0, \infty)$ . For a query point  $\mathbf{x} \in \mathcal{X}$ , let  $\mathcal{N}_k(\mathbf{x})$  denote the indices of the  $k$  training points closest to  $\mathbf{x}$  under  $d$ . The  **$k$ -nearest neighbour classifier** predicts

$$\hat{f}(\mathbf{x}) = \arg \max_{c \in \mathcal{Y}} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c].$$

That is, the predicted class is the plurality vote among the  $k$  nearest neighbours. For regression, the prediction is the mean of the  $k$  nearest target values:

$$\hat{f}(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i.$$

The most common choice of distance is the Euclidean metric,  $d(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_2$ . The Manhattan metric  $d(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_1$  and the general Minkowski metric  $\|\mathbf{x} - \mathbf{x}'\|_r$  are also used; different metrics assign different neighbourhood shapes and can matter when features have very different ranges or when the geometry of the problem is non-isotropic.

*Remark 3.2.* Because  $k$ -NN uses raw feature distances, it is sensitive to the scale of individual features. A feature measured in thousands (e.g. annual salary) will dominate the Euclidean distance

over a feature measured in units (e.g. number of children), regardless of their relative importance to the prediction task. Standardising features to zero mean and unit variance before applying  $k$ -NN is standard practice, and one reason the transformation methods of Weeks 3 and 4 matter directly for model performance.

### 3.2 Decision boundaries

The **decision boundary** of a classifier is the set of points  $\mathbf{x}$  at which the predicted class changes. For  $k$ -NN with  $k = 1$  in two dimensions, the *Voronoi diagram* of the training points partitions the plane into regions, one per training point, such that each region contains all points closer to that training point than to any other; points in a region receive the label of the corresponding training point. The classification decision boundary is not the entire Voronoi diagram, however: it consists only of those Voronoi boundaries that separate two cells whose training points carry different class labels. A boundary between two adjacent cells belonging to the same class is a Voronoi edge but not a decision boundary, since the predicted label does not change across it.

As  $k$  increases, predictions at each point are determined by a larger set of neighbours, and the decision boundary becomes smoother and less sensitive to individual training points. For  $k = n$ , the prediction is the same everywhere – the majority class over  $\mathcal{D}$  – and the decision boundary disappears. This progression from a jagged, data-hugging boundary (small  $k$ ) to a flat, uninformative one (large  $k$ ) is the geometric manifestation of the bias–variance trade-off.

**Example 3.3.** Consider a binary classification problem in  $\mathbb{R}^2$  where class 1 occupies a roughly circular region in the centre of the feature space and class 0 surrounds it. With  $k = 1$ , the decision boundary closely tracks the training points; a noisy label anywhere near the true boundary will create a small island of the wrong class. With  $k = 15$ , the boundary is smooth and approximately circular, ignoring isolated noisy labels. With  $k = 51$  (more than half the training points), the boundary may have moved entirely to the wrong side of the true boundary if the outer class 0 region contains more training points overall. The optimal  $k$  in this example is somewhere in between, and cross-validation is needed to find it.

### 3.3 The role of $k$ and the bias–variance trade-off

The integer  $k$  is a *hyperparameter*: it is chosen before training, not learned from  $\mathcal{D}$ . Its value controls the effective complexity of the classifier.

When  $k = 1$ , the prediction at any query point equals the label of its single nearest training neighbour. The classifier memorises  $\mathcal{D}$  exactly and achieves zero training error. But this comes at a cost: if a label is corrupted by noise, the corruption is reproduced faithfully for all queries near that point. The prediction is highly variable across different realisations of  $\mathcal{D}$ , because the identity of the single nearest neighbour can change substantially when even a few training points are replaced.

As  $k$  increases, predictions are averaged over a larger neighbourhood. Random label noise in individual training points has less effect, so variance falls. At the same time, the neighbourhood covers a larger region of  $\mathcal{X}$ , and fine local structure in  $f^*$  is smoothed away; bias rises. In the limit  $k = n$ , the prediction at every query point is the same constant – the overall majority class – so it no longer varies with  $\mathbf{x}$ ; however, that constant is itself computed from  $\mathcal{D}$ , and it changes if  $\mathcal{D}$  is resampled, so this limit does not have zero variance in the bias–variance sense. It does typically have high bias when  $f^*$  is not constant, since a single overall vote cannot capture any local structure.

The bias–variance decomposition makes this precise. Averaging over  $k$  independent noise terms contributes approximately  $\sigma^2/k$  to the conditional variance of the  $k$ -NN regressor at a fixed query

point  $\mathbf{x}$ , given the identity of its neighbours; this falls as  $k$  grows. The total sampling variance also reflects randomness in which points happen to fall in  $\mathcal{N}_k(\mathbf{x})$  and variation in  $f^*$  across that neighbourhood, so  $\sigma^2/k$  should be read as the noise-averaging contribution rather than the whole variance term. The bias depends on how much  $f^*$  varies over the  $k$ -neighbourhood and increases as the neighbourhood grows. The total expected squared error is minimised at an intermediate  $k$  that balances these effects.

**Proposition 3.4** (Universal consistency of  $k$ -NN). *For  $k$ -NN classification on  $\mathbb{R}^p$  with Euclidean distance, suppose  $k \rightarrow \infty$  and  $k/n \rightarrow 0$  as  $n \rightarrow \infty$ . Then the risk  $R_n$  of the  $k$ -NN classifier under 0–1 loss satisfies*

$$R_n \rightarrow R^*$$

*for every distribution  $P$  on  $\mathbb{R}^p \times \mathcal{Y}$ . That is,  $k$ -NN is universally consistent on  $\mathbb{R}^p$ : with  $k$  and  $n$  growing at this relative rate, its risk converges to the Bayes risk regardless of the underlying distribution.*

*Remark 3.5.* This result is a consequence of Stone’s (1977) general consistency theorem for local averaging estimators. Stone’s paper is stated for nonparametric regression – for a weighted local average  $\hat{\eta}_n(\mathbf{x})$  of the regression function  $\eta(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]$ , satisfying certain conditions on the weights – but the  $k$ -NN classification rule is obtained by applying it to a  $\{0,1\}$ -valued target and thresholding the resulting estimate at  $1/2$ ; consistency of the regression estimate then implies consistency of the induced classifier. The proposition above is this application of Stone’s framework to  $k$ -NN classification, not a direct restatement of the paper’s own theorem.

*Remark 3.6* (The classical fixed- $k$  case). Before Stone’s general result, Cover and Hart (1967) analysed the simpler case of a *fixed*  $k$  (in particular  $k = 1$ ) as  $n \rightarrow \infty$ . In that regime the risk does *not* converge to  $R^*$  in general; instead, for binary classification,

$$R^* \leq \lim_{n \rightarrow \infty} R_{1\text{-NN}} \leq 2R^*(1 - R^*) \leq 2R^*.$$

(For  $C > 2$  classes the corresponding bound takes a different,  $C$ -dependent form.) This is a genuinely different statement from Stone’s theorem above: with  $k$  fixed, a query point’s nearest neighbour becomes arbitrarily close as  $n \rightarrow \infty$ , but its label is still a single noisy draw, so the asymptotic risk is bounded relative to  $R^*$  without generally equalling it. Universal consistency requires  $k$  to grow as well, so that the local vote averages out label noise; this is precisely the extra condition in Stone’s theorem.

Universal consistency is a strong guarantee, but it is asymptotic. For the finite sample sizes typical in practice,  $k$ -NN suffers from the curse of dimensionality discussed in Week 4: in high dimensions, the volume of the neighbourhood required to contain  $k$  points grows rapidly with  $p$ , and the notion of a local neighbourhood becomes vacuous. A neighbourhood that is geometrically local in  $\mathbb{R}^2$  may span the entire feature space in  $\mathbb{R}^{100}$ . This is one reason parametric models, which impose structure on  $f^*$  rather than inferring it from local geometry, are preferred when  $p$  is large relative to  $n$ .

### 3.4 Weighted $k$ -NN

A natural extension is to weight the contribution of each neighbour by its distance to the query point, so that closer neighbours have more influence. The weighted  $k$ -NN classifier predicts

$$\hat{f}(\mathbf{x}) = \arg \max_{c \in \mathcal{Y}} \sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i \mathbf{1}[y_i = c], \quad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i) + \epsilon},$$

for a small constant  $\epsilon > 0$ , with analogous modification for regression. The  $\epsilon$  term is needed because  $w_i$  is otherwise undefined when the query point coincides exactly with a training point; a common convention in that case is instead to predict directly from the exactly-matching neighbour(s). Distance weighting reduces the sensitivity of the prediction to the exact value of  $k$ , because a neighbour just inside the neighbourhood boundary has little weight, though it does not by itself resolve a tie in *which* points fall within the  $k$ -th-neighbour boundary in the first place.

### 3.5 Computational considerations

A naïve implementation of  $k$ -NN computes the distance from the query point to every training point, requiring  $O(np)$  operations per query. For large  $n$  or  $p$  this is prohibitive. Data structures such as  $k$ -d trees and ball trees partition  $\mathcal{X}$  hierarchically and allow nearest-neighbour search without visiting every point. A  $k$ -d tree recursively splits the data along coordinate axes and achieves average-case  $O(p \log n)$  query time under favourable conditions, though this degrades as  $p$  grows; a ball tree splits by enclosing balls and can outperform a  $k$ -d tree on some higher-dimensional or structured data, but neither is uniformly better in high dimensions, and the best choice depends on the data. Both are available in `sklearn.neighbors`, whose default 'auto' setting selects between `ball_tree`, `kd_tree`, and brute force based on the shape and structure of the data. For very large datasets or approximate answers, locality-sensitive hashing (LSH) provides sub-linear query times at the cost of occasional errors.

## 4 Choosing $k$ by Cross-Validation

Cross-validation is the standard method for selecting hyperparameters such as  $k$ . The central idea is to use only the training data to simulate how the model will perform on unseen examples, thereby avoiding the need to use the test set during model selection.

**Definition 4.1.** In  **$m$ -fold cross-validation**, the training set  $\mathcal{D}$  is partitioned randomly into  $m$  folds  $\mathcal{D}_1, \dots, \mathcal{D}_m$  of approximately equal size. For each fold  $j \in \{1, \dots, m\}$ , a model is trained on  $\mathcal{D} \setminus \mathcal{D}_j$  and its empirical risk is evaluated on the held-out fold  $\mathcal{D}_j$ . The **cross-validation estimate** of the generalisation error for hyperparameter value  $k$  is

$$\widehat{\text{CV}}(k) = \frac{1}{m} \sum_{j=1}^m \hat{R}_j(\hat{f}_{k, \mathcal{D} \setminus \mathcal{D}_j}),$$

where  $\hat{R}_j$  denotes the empirical risk on fold  $j$ .

The hyperparameter value minimising  $\widehat{\text{CV}}(k)$  is selected, and a final model is then trained on the full  $\mathcal{D}$  using that value. The final model is what is used for prediction; the cross-validation procedure is only used to select  $k$ .

The special case  $m = n$  is called **leave-one-out cross-validation** (LOOCV). At each step, one training point is held out, the model is trained on the remaining  $n - 1$  points, and the prediction is evaluated at the held-out point. LOOCV uses the maximum possible training data at each step and produces an estimate with low bias, since each training set differs from  $\mathcal{D}$  by only one point. For  $k$ -NN specifically, LOOCV has a convenient structure: the single held-out point  $(\mathbf{x}_j, y_j)$  is its own nearest neighbour, so it must be excluded from the neighbourhood when predicting its own label, but otherwise the distances to all other points are unchanged. This means LOOCV for  $k$ -NN can be computed without refitting from scratch for each fold.

The practical choice between  $m$ -fold CV and LOOCV involves a trade-off between bias, variance, and computational cost, and depends on  $n$  as well as on the model. LOOCV may be attractive when  $n$  is small, since it uses nearly all the data for training at each step and has low bias, but its estimate can have relatively high variance because the  $n$  training sets overlap heavily with one another. For moderate to large  $n$ , 5-fold or 10-fold CV is standard: 10-fold uses 90% of the data for training at each step and produces a reasonably accurate estimate at a fraction of the cost of LOOCV. There is some evidence that  $m$ -fold CV with moderate  $m$  can have lower variance than LOOCV for unstable estimators, because the training sets share fewer points in common and their errors are less correlated.

*Remark 4.2.* Cross-validation estimates the error of the learning *procedure*, not of the specific model trained on all of  $\mathcal{D}$ . Strictly speaking,  $\widehat{\text{CV}}(k)$  estimates the generalisation error of a model trained on  $(1 - 1/m)n$  points rather than  $n$  points. For large  $n$  this difference is negligible, but for small  $n$  it means cross-validation tends to overestimate the error of the final model.

#### 4.1 The one-standard-error rule

In practice, the cross-validation curve  $\widehat{\text{CV}}(k)$  plotted against  $k$  is often relatively flat near its minimum, with several values of  $k$  giving nearly identical estimated errors. Because the minimum is estimated with noise, choosing the exact minimiser can lead to a more complex model than necessary. The **one-standard-error rule** addresses this: compute the standard error of  $\widehat{\text{CV}}(k)$  across the  $m$  folds,

$$\widehat{\text{SE}}(k) = \frac{1}{\sqrt{m}} \text{std}_j \left( \hat{R}_j \left( \hat{f}_{k, \mathcal{D} \setminus \mathcal{D}_j} \right) \right),$$

and select the simplest model (largest  $k$  for  $k$ -NN, since larger  $k$  gives a smoother, less complex classifier) whose cross-validation error lies within one standard error of the minimum. This tends to produce models that generalise at least as well as the nominal minimiser while being less prone to overfitting the selection criterion itself.

## 5 Exercises

**Exercise 5.1.** Let  $\mathcal{D}$  consist of six points in  $\mathbb{R}^2$  with binary labels:

$$\begin{aligned} \mathbf{x}_1 &= (1, 2), y_1 = 0; & \mathbf{x}_2 &= (2, 3), y_2 = 0; & \mathbf{x}_3 &= (3, 1), y_3 = 0; \\ \mathbf{x}_4 &= (6, 5), y_4 = 1; & \mathbf{x}_5 &= (7, 4), y_5 = 1; & \mathbf{x}_6 &= (5, 6), y_6 = 1. \end{aligned}$$

For the query point  $\mathbf{q} = (4, 4)$ , compute the Euclidean distance from  $\mathbf{q}$  to each training point and determine the prediction of the  $k$ -NN classifier for  $k = 1$ ,  $k = 3$ , and  $k = 5$ . Identify any ties and state how they affect the prediction.

*Solution.* The squared distances are  $(4 - x_1)^2 + (4 - x_2)^2$  for each point:

$$\begin{aligned} d(\mathbf{q}, \mathbf{x}_1) &= \sqrt{(3)^2 + (2)^2} = \sqrt{13} \approx 3.61, \\ d(\mathbf{q}, \mathbf{x}_2) &= \sqrt{(2)^2 + (1)^2} = \sqrt{5} \approx 2.24, \\ d(\mathbf{q}, \mathbf{x}_3) &= \sqrt{(1)^2 + (3)^2} = \sqrt{10} \approx 3.16, \\ d(\mathbf{q}, \mathbf{x}_4) &= \sqrt{(2)^2 + (1)^2} = \sqrt{5} \approx 2.24, \\ d(\mathbf{q}, \mathbf{x}_5) &= \sqrt{(3)^2 + (0)^2} = 3, \\ d(\mathbf{q}, \mathbf{x}_6) &= \sqrt{(1)^2 + (2)^2} = \sqrt{5} \approx 2.24. \end{aligned}$$

Ordering by distance:  $\mathbf{x}_2$  (class 0),  $\mathbf{x}_4$  (class 1), and  $\mathbf{x}_6$  (class 1) are all equidistant at  $\sqrt{5}$ ; next is  $\mathbf{x}_5$  at distance 3, then  $\mathbf{x}_3$  at  $\sqrt{10}$ , then  $\mathbf{x}_1$  at  $\sqrt{13}$ .

For  $k = 1$ : any one of  $\mathbf{x}_2$ ,  $\mathbf{x}_4$ ,  $\mathbf{x}_6$  could be selected as the single nearest neighbour. Without a tie-breaking rule, the prediction is not uniquely defined. A practical implementation must therefore specify a deterministic tie-breaking rule; different rules may give different  $k = 1$  predictions here – for instance, breaking ties toward  $\mathbf{x}_2$  would predict class 0, while breaking ties toward  $\mathbf{x}_4$  or  $\mathbf{x}_6$  would predict class 1.

For  $k = 3$ : the three nearest points are  $\mathbf{x}_2$  (class 0),  $\mathbf{x}_4$  (class 1), and  $\mathbf{x}_6$  (class 1). The vote is 2–1 in favour of class 1; the classifier predicts class 1.

For  $k = 5$ : the five nearest are  $\mathbf{x}_2$  (0),  $\mathbf{x}_4$  (1),  $\mathbf{x}_6$  (1),  $\mathbf{x}_5$  (1),  $\mathbf{x}_3$  (0). The vote is 3–2 in favour of class 1; the classifier predicts class 1.  $\square$

**Exercise 5.2.** A  $k$ -NN regressor is used to predict a continuous target from a single feature  $x \in [0, 1]$ , with training set  $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{20}$ , where the  $x_i$  are spaced roughly evenly over  $[0, 1]$  and  $y_i = \sin(2\pi x_i) + \varepsilon_i$  with  $\varepsilon_i \sim N(0, 0.1^2)$ . Without carrying out any computation, describe qualitatively what the fitted function  $\hat{f}(x)$  looks like for  $k = 1$ ,  $k = 5$ , and  $k = 20$ . For each value of  $k$ , state whether bias or variance is the dominant source of error and explain why. How would your answer change if the noise level were increased to  $\sigma = 0.5$ ?

*Solution.* For  $k = 1$ , the fitted function is piecewise constant, jumping between the target values of individual training points. It passes through every training observation exactly and will reproduce any label noise faithfully. Variance dominates: different training sets will yield very different fitted functions, since replacing even one point changes the prediction in its neighbourhood. Bias is low because the function can track the true sinusoid closely between training points, given enough data. For  $k = 5$ , the prediction at each query point averages over five neighbours. The fitted function is smoother – still piecewise constant in the  $k$ -NN sense, but with fewer and less extreme jumps – and closely approximates  $\sin(2\pi x)$  over most of  $[0, 1]$ . Bias and variance are more balanced: noise in individual labels is partially averaged out (lower variance than  $k = 1$ ), but the neighbourhood is small enough to track the sinusoidal shape (lower bias than  $k = 20$ ).

For  $k = 20 = n$ , the prediction is the same everywhere: the mean of all 20 target values. Since the  $x_i$  are roughly evenly spaced over  $[0, 1]$  and  $\sin(2\pi x)$  integrates to zero over that interval, this sample mean is approximately zero. The fitted function is therefore close to a horizontal line at zero, utterly failing to capture the sinusoidal structure. Bias dominates: the average prediction is systematically wrong everywhere except near the zero crossings of the sinusoid. For a fixed training set the prediction does not vary with  $x$ , but the sample mean itself is a statistic of  $\mathcal{D}$  and would change under a different training sample, so its sampling variance is small in this setting but not exactly zero.

If  $\sigma$  is increased to 0.5, the irreducible noise term rises substantially. The relative cost of high variance also increases: with noisier labels,  $k = 1$  becomes even more problematic because it memorises large errors rather than small ones. In this smooth, low-dimensional setting, the optimal  $k$  would generally be expected to shift upward, because averaging over more neighbours can reduce the increased noise variance, and one would expect the cross-validation curve to shift accordingly. The bias of  $k = 5$  may become acceptable relative to its now-cheaper variance reduction, while  $k = 20$  remains hopeless since no averaging cures its fundamental bias.  $\square$

**Exercise 5.3.** Suppose you run 5-fold cross-validation on a  $k$ -NN classifier for  $k \in \{1, 3, 5, 7, 9\}$  and obtain the following cross-validation errors and estimated standard errors:

| $k$ | $\widehat{CV}(k)$ | $\widehat{SE}(k)$ |
|-----|-------------------|-------------------|
| 1   | 0.24              | 0.04              |
| 3   | 0.18              | 0.03              |
| 5   | 0.16              | 0.03              |
| 7   | 0.17              | 0.03              |
| 9   | 0.19              | 0.04              |

Apply the one-standard-error rule to select a value of  $k$ . State the minimum cross-validation error and the corresponding standard error, identify all values of  $k$  within one standard error of the minimum, and select the value of  $k$  that the rule recommends.

*Solution.* The minimum cross-validation error is  $\widehat{CV}(5) = 0.16$  with  $\widehat{SE}(5) = 0.03$ . The one-standard-error threshold is  $0.16 + 0.03 = 0.19$ .

The values of  $k$  with  $\widehat{CV}(k) \leq 0.19$  are  $k = 3$  (error 0.18),  $k = 5$  (error 0.16),  $k = 7$  (error 0.17), and  $k = 9$  (error 0.19). The value  $k = 1$  has error 0.24, which exceeds the threshold.

Among the qualifying values, the one-standard-error rule selects the simplest model, i.e. the largest  $k$  (since larger  $k$  gives a less complex, smoother classifier). That is  $k = 9$ . Despite  $k = 5$  having the nominally lowest cross-validation error,  $k = 9$  achieves essentially the same performance – within the estimation noise of the cross-validation procedure – while being a more parsimonious model less likely to overfit the selection criterion.  $\square$

## Further Reading

Lindholm, A., Wahlström, N., Lindsten, F., and Schön, T. B. (2022). *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press. The early chapters introduce  $k$ -nearest neighbours alongside decision trees as a first, non-parametric approach to supervised learning, and cover cross-validation and the bias–variance trade-off at a level consistent with this lecture.

James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. Chapter 2 introduces the bias–variance trade-off and the  $K$ -nearest neighbours classifier together, and Chapter 5 covers cross-validation and resampling methods more generally; the one-standard-error rule of Section 4.1 above is a widely used model-selection heuristic.

Stone, C. J. (1977). Consistent nonparametric regression. *Annals of Statistics*, 5(4), 595–620. Stone’s paper establishes universal consistency for a general class of local averaging (nonparametric regression) estimators; the  $k$ -NN classification result of Section 3.3 is a consequence of this framework, obtained by applying it to a 0/1-valued target and thresholding at 1/2, rather than a theorem stated directly for classification in the original paper.

Cover, T., and Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27. The classical analysis of the fixed- $k$  (in particular 1-NN) case discussed in the remark following Stone’s theorem.

## References

Lindholm, A., Wahlström, N., Lindsten, F. and Schön, T.B., 2022. *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press.

- James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J., 2023. *An Introduction to Statistical Learning with Applications in Python*. Springer.
- Stone, C.J., 1977. Consistent nonparametric regression. *The Annals of Statistics*, 5(4), pp.595–620.
- Cover, T. and Hart, P., 1967. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), pp.21–27.